

Placement Empowerment Program
Cloud Computing and DevOps Centre

Use Cloud Storage

Create a storage bucket on your cloud platform and upload/download files. Configure access permissions for the bucket.

Name: Neithal Pillai

Department : IT

Introduction and Overview

Deploying a Flask web application on an AWS EC2 instance allows for cloud-based hosting of applications with better scalability and availability. This PoC demonstrates the step-by-step process of setting up a Flask application, configuring security settings, and running the app to be accessible from the internet.

Objective

The goal of this PoC is to:

1. Set up an AWS EC2 instance.
2. Configure necessary security settings.
3. Install dependencies and set up a virtual environment.
4. Deploy a Flask application on the instance.
5. Ensure accessibility over the internet.

Importance

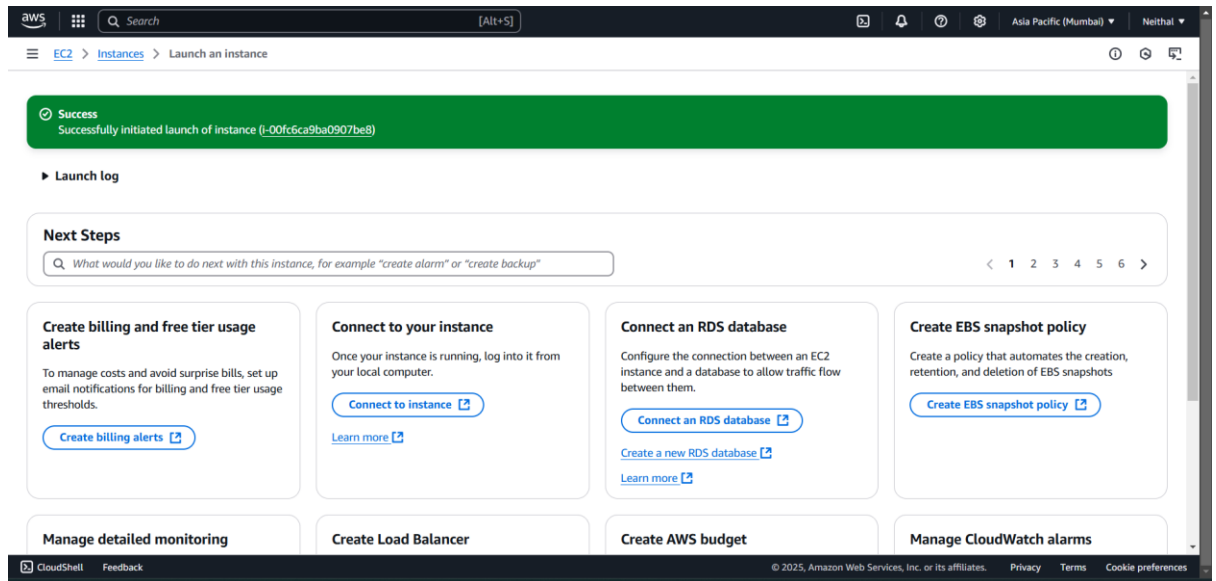
Hosting a web application on a cloud platform like AWS provides multiple advantages, including high availability, scalability, and ease of deployment. By deploying Flask on an EC2 instance, we gain a flexible and cost-effective way to host web applications.

Steps for Execution

Step 1: Launch an AWS EC2 Instance

- Access the AWS Management Console and navigate to the EC2 dashboard.
- Initiate the process to launch a new instance and select an appropriate Amazon Machine Image (AMI), preferably Ubuntu.
- Choose an instance type suitable for the workload, such as t2.micro for free-tier eligibility.
- Configure necessary details such as instance name, storage, and network settings.

- Set up a security group to allow SSH (port 22) and HTTP access (port 5000).
- Generate and download the private key file (.pem) for secure SSH authentication.
- Complete the launch process and note down the assigned public IP address.



Step 2: Establish SSH Connection to the EC2 Instance

- Open a terminal and navigate to the directory where the .pem key file is stored.
- Connect securely to the EC2 instance using SSH by specifying the key file and public IP address.
- Verify the connection to ensure successful access to the instance.

```

ubuntu@ip-172-31-9-255: ~
neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ ls -l *.pem
-rw-r--r-- 1 neith 197609 1678 Feb 23 22:49 keyforIAM.pem
-rw-r--r-- 1 neith 197609 1678 Mar  3 08:30 keyforflasknew.pem
-rw-r--r-- 1 neith 197609 1674 Feb 23 19:25 newkey2.pem

neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ ssh -i keyforflasknew.pem user@13.232.139.217
The authenticity of host '13.232.139.217 (13.232.139.217)' can't be established.
ED25519 key fingerprint is SHA256:vK1vj6SZvrd9I0m7UoVOLXmLKOFhLexxV0evqbjmFik.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.232.139.217' (ED25519) to the list of known hosts
.
user@13.232.139.217: Permission denied (publickey).

neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ ^C

neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ ssh -i keyforflasknew.pem ubuntu@13.232.139.217
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-1021-aws x86_64)

```

Step 3: Install Essential Packages and Dependencies

- Update the package list and upgrade existing system packages to the latest versions.
- Install Python along with necessary utilities for managing dependencies and virtual environments.
- Confirm successful installation by checking the Python version and package manager functionality.

```

ubuntu@ip-172-31-9-255: ~
Preparing to unpack .../libudev1_255.4-1ubuntu8.5_amd64.deb ...
Unpacking libudev1:amd64 (255.4-1ubuntu8.5) over (255.4-1ubuntu8.4) ...
Setting up libudev1:amd64 (255.4-1ubuntu8.5) ...
(Reading database ... 70608 files and directories currently installed.)
Preparing to unpack .../libmount1_2.39.3-9ubuntu6.2_amd64.deb ...
Unpacking libmount1:amd64 (2.39.3-9ubuntu6.2) over (2.39.3-9ubuntu6.1) ...
Setting up libmount1:amd64 (2.39.3-9ubuntu6.2) ...
(Reading database ... 70608 files and directories currently installed.)
Preparing to unpack .../libdevmapper1.02.1_2%3a1.02.185-3ubuntu3.2_amd64.deb ...
Unpacking libdevmapper1.02.1:amd64 (2:1.02.185-3ubuntu3.2) over (2:1.02.185-3ubuntu3.1) ...
Preparing to unpack .../libuuid1_2.39.3-9ubuntu6.2_amd64.deb ...
Unpacking libuuid1:amd64 (2.39.3-9ubuntu6.2) over (2.39.3-9ubuntu6.1) ...
Setting up libuuid1:amd64 (2.39.3-9ubuntu6.2) ...
(Reading database ... 70608 files and directories currently installed.)
Preparing to unpack .../libcryptsetup12_2%3a2.7.0-1ubuntu4.2_amd64.deb ...
Unpacking libcryptsetup12:amd64 (2:2.7.0-1ubuntu4.2) over (2:2.7.0-1ubuntu4.1) ...
Preparing to unpack .../libfdisk1_2.39.3-9ubuntu6.2_amd64.deb ...
Unpacking libfdisk1:amd64 (2.39.3-9ubuntu6.2) over (2.39.3-9ubuntu6.1) ...
Preparing to unpack .../rsync_3.2.7-1ubuntu1.2_amd64.deb ...
Unpacking rsync (3.2.7-1ubuntu1.2) over (3.2.7-1ubuntu1.1) ...
Progress: [ 20%] [#####.....]

```

Step 4: Create and Configure a Virtual Environment

- Initialize a Python virtual environment within the instance.
- Activate the virtual environment to ensure all dependencies remain isolated from system-wide installations.
- Install Flask and other required libraries within the activated environment.

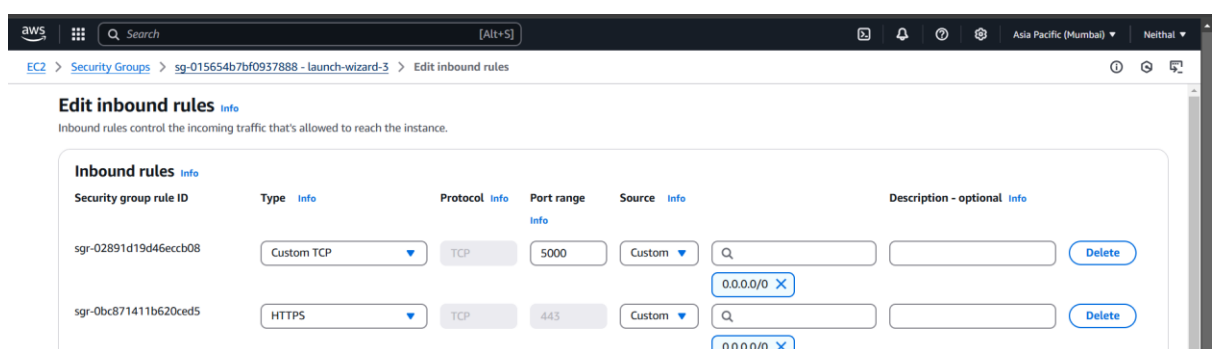
```
No containers need to be restarted.

User sessions running outdated binaries:
ubuntu @ session #3: sshd[1045,1155]
ubuntu @ user manager service: systemd[1050]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-9-255:~$ ^C
ubuntu@ip-172-31-9-255:~$ python3 -m venv flask_env
ubuntu@ip-172-31-9-255:~$ source flask_env/bin/activate
(flask_env) ubuntu@ip-172-31-9-255:~$
```

Step 5: Configure Security Groups and Firewall Rules

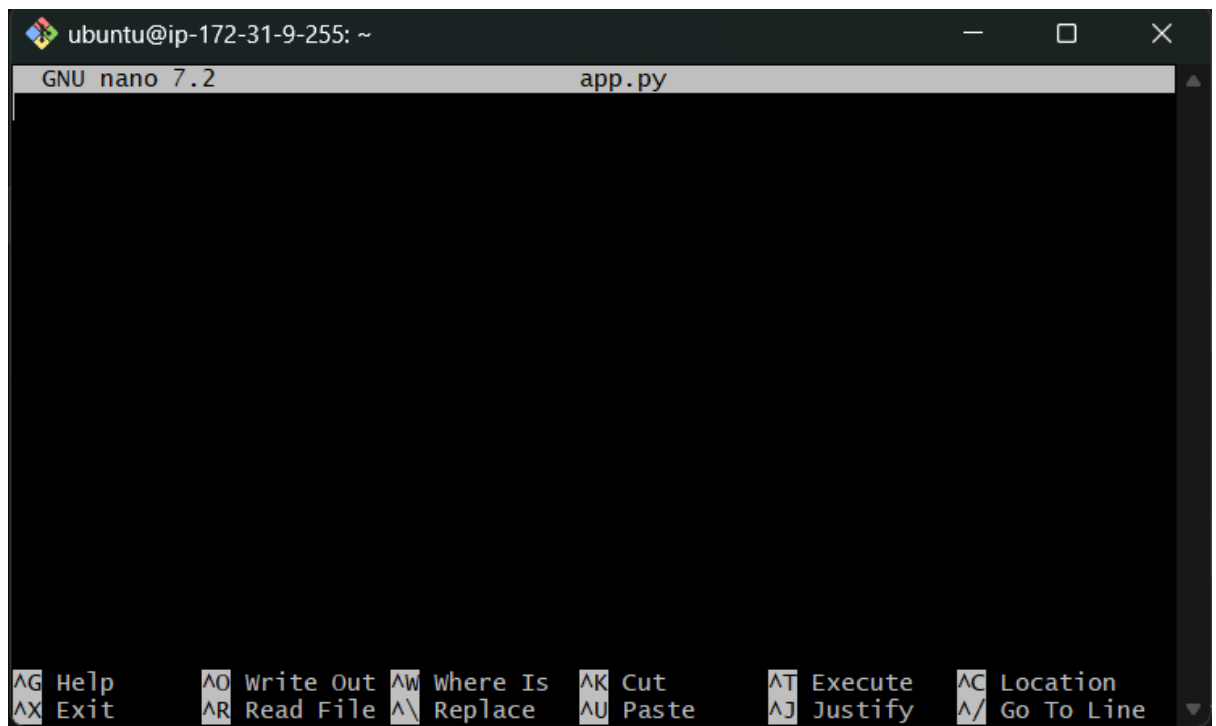
- Verify the status of the firewall configuration on the EC2 instance.
- If firewall restrictions are in place, explicitly allow inbound traffic on port 5000 to enable external access.
- Modify AWS Security Group rules to permit HTTP traffic on the designated application port.
- Confirm that the instance is accessible from external networks.



Step 6: Deploy the Flask Application and Start the Server

- Ensure the application is correctly placed within the instance directory structure.

- Start the Flask application, ensuring it listens on all available network interfaces.
- Verify that the server is running without errors and is accessible from within the instance.



The image shows a terminal window titled 'ubuntu@ip-172-31-9-255: ~'. Inside the terminal, the GNU nano 7.2 text editor is open, editing a file named 'app.py'. The editor's interface is dark-themed, with a light gray header bar showing the editor name and file name. The main editing area is black. At the bottom, a status bar displays various keyboard shortcuts for nano, such as ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^C Location, ^X Exit, ^R Read File, ^\ Replace, ^U Paste, ^J Justify, and ^_ Go To Line.

Step 7: Test Application Accessibility from a Web Browser

- Open a web browser and enter the instance's public IP address followed by the designated port.
- Confirm that the Flask application loads correctly and responds to HTTP requests.
- If necessary, troubleshoot any connectivity or permission issues.



Step 8: Implement a Production WSGI Server (Optional)

- Install a production-grade WSGI server, such as gunicorn, for enhanced stability.
- Configure and run the Flask application using the WSGI server to handle concurrent requests efficiently.
- Optimize the deployment by adjusting server settings and monitoring resource utilization.

Conclusion

This PoC successfully demonstrates the deployment of a Flask web application on AWS EC2, highlighting the necessary configurations and security measures. By following these structured steps, Flask applications can be effectively hosted and made accessible over the internet.

What I Learned

1. Configuring and launching an AWS EC2 instance for hosting applications.
2. Establishing secure SSH connections and managing cloud-based virtual environments.
3. Installing and maintaining Python dependencies within a controlled environment.
4. Deploying and running a Flask application on a cloud-based server.
5. Implementing security measures to ensure safe and reliable access to the application.

6. Using a production-ready WSGI server to optimize application performance.